GUEST LECTURE REPORT

Databricks and Spark Programming

Big Data Technology Lab (BDTL)

Particular	Information
Student Name	Adesh Patankar
Subject	Big Data Technology Lab (BDTL)
University / Department	MIT-ADT University, Pune – MIT School of Computing
Guest Lecture Topic	Databricks and Spark Programming
Resource Person	Dr. Darshan Ingle – Principal Consultant, Senior Data Scientist and Corporate Trainer
Date and Time	Saturday, 10 October 2026, 1:30 PM to 3:30 PM

1. Introduction

The session on Databricks and Spark Programming, conducted by Dr. Darshan Ingle, introduced the role of distributed computing in handling large-scale data. Traditional systems that rely on a single machine can struggle when datasets become very large. Apache Spark addresses this challenge by distributing processing across partitions, while Databricks provides an integrated environment for data engineering and analytics. The lecture connected the underlying concepts with practical PySpark examples demonstrated in a Google Colab notebook using a local Spark session.

2. Main Objectives of the Session

- Learn how Spark organizes work through applications, jobs, stages, tasks and partitions.
- Understand how distributed processing allows several parts of a dataset to be processed concurrently.
- Study lazy evaluation and the difference between transformations and actions.
- Practice common DataFrame operations such as select(), filter(), withColumn(), groupBy() and aggregations.
- Identify performance issues caused by collecting excessive data to the driver and by unnecessary data shuffling.

3. Connection with the BDTL Subject

The lecture directly supported the Big Data Technology Lab syllabus, particularly its focus on cluster computing, scalable processing and data pipelines. Spark's in-memory execution model and fault-tolerant data abstractions make it useful for processing information at scale. The examples showed how transformations, partitioning and execution planning influence the efficiency of a data engineering workflow. These ideas are important when developing ETL pipelines and other distributed applications.

TECHNICAL CONCEPTS AND DEMONSTRATIONS

4. Spark Execution Model

Apache Spark divides data into partitions and schedules work across available worker resources. The resource person explained the relationship between the main levels of execution: an application contains the overall Spark program; an action can initiate a job; a job is broken into stages; and each stage contains tasks that operate on partitions. A partition represents a portion of the data, whereas a task is the unit of work performed on that portion.

APPLICATION	JOB	STAGE	TASK	PARTITION
Complete Spark program	Triggered by action	Separated by shuffle	Work on partition	Data segment

Figure 1. Simplified hierarchy of Spark execution components.

The number of executor cores affects how many tasks can run at the same time. For instance, when a stage contains eight partitions and four executor cores are available, Spark can usually execute four tasks concurrently in one wave, followed by the remaining four.

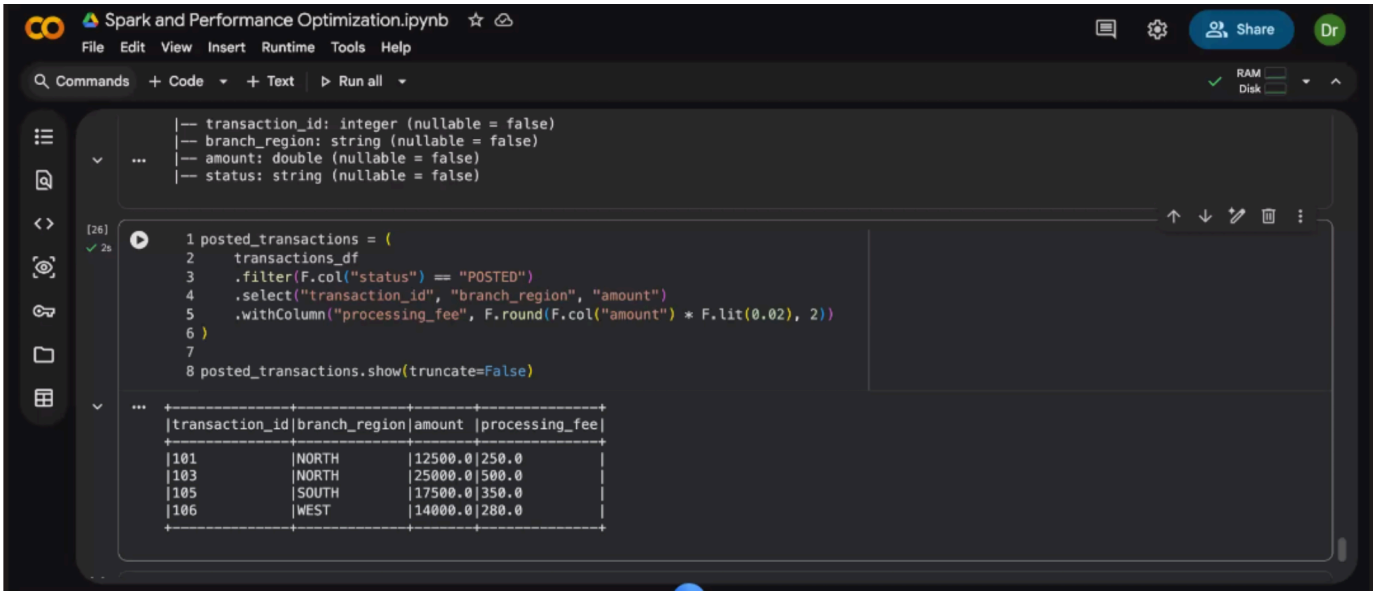
5. Lazy Evaluation and DataFrame Operations

A key concept covered in the session was lazy evaluation. Transformations do not immediately process all records. Instead, Spark records the requested operations as a plan and waits until an action asks for a result. With the full chain of operations available, Spark can optimize the plan and avoid performing work that is not required for the requested output.

Operation	Purpose demonstrated
select()	Chooses the columns needed for further processing.
filter()	Retains only records that meet a condition, such as transactions with status “POSTED”.
withColumn()	Creates or updates a column; the example calculated a processing fee equal to 2% of the amount, rounded to two decimal places.
groupBy() with aggregations	Produces grouped summaries, such as transaction counts and totals for each category.
mapPartitionsWithIndex()	Makes partition-level distribution easier to inspect in a small RDD example.

6. Practical Notebook Work

The live notebook examples applied filtering, column selection and derived-column creation to transaction data. The demonstration also inspected partition contents and considered how Spark builds a plan before running it. These activities helped connect the API syntax with the way Spark distributes and executes data-processing work.



OPTIMISATION, LEARNING AND CONCLUSION

7. Performance and Scalability

The session highlighted the risks of using collect() on a large dataset. In the example discussed, the data represented a bank with ten lakh transactions. Calling collect() attempts to transfer all records to the driver program, which can consume too much driver memory and may cause the application to fail. For limited inspection, the lecture identified show(n) and take(n) as safer alternatives.

The speaker also explained that operations such as joins and groupings may trigger a shuffle. During a shuffle, records are redistributed across the cluster according to a key, which adds communication and processing overhead. The illustrated physical plan included an Exchange operation using hashpartitioning(account_id, 32). Adaptive Query Execution settings—covering adaptive execution, partition coalescing and skew-join handling—were also discussed as ways to manage execution behaviour. Filtering data earlier and choosing partitions carefully can help reduce unnecessary work.

8. Wider Uses of Spark and Databricks

Although the practical portion concentrated on batch processing with DataFrames and execution plans, the same Spark ecosystem also supports Structured Streaming for streaming workloads, MLlib for distributed machine learning, and lakehouse-style data architectures available through Databricks.

9. Key Learning Outcomes

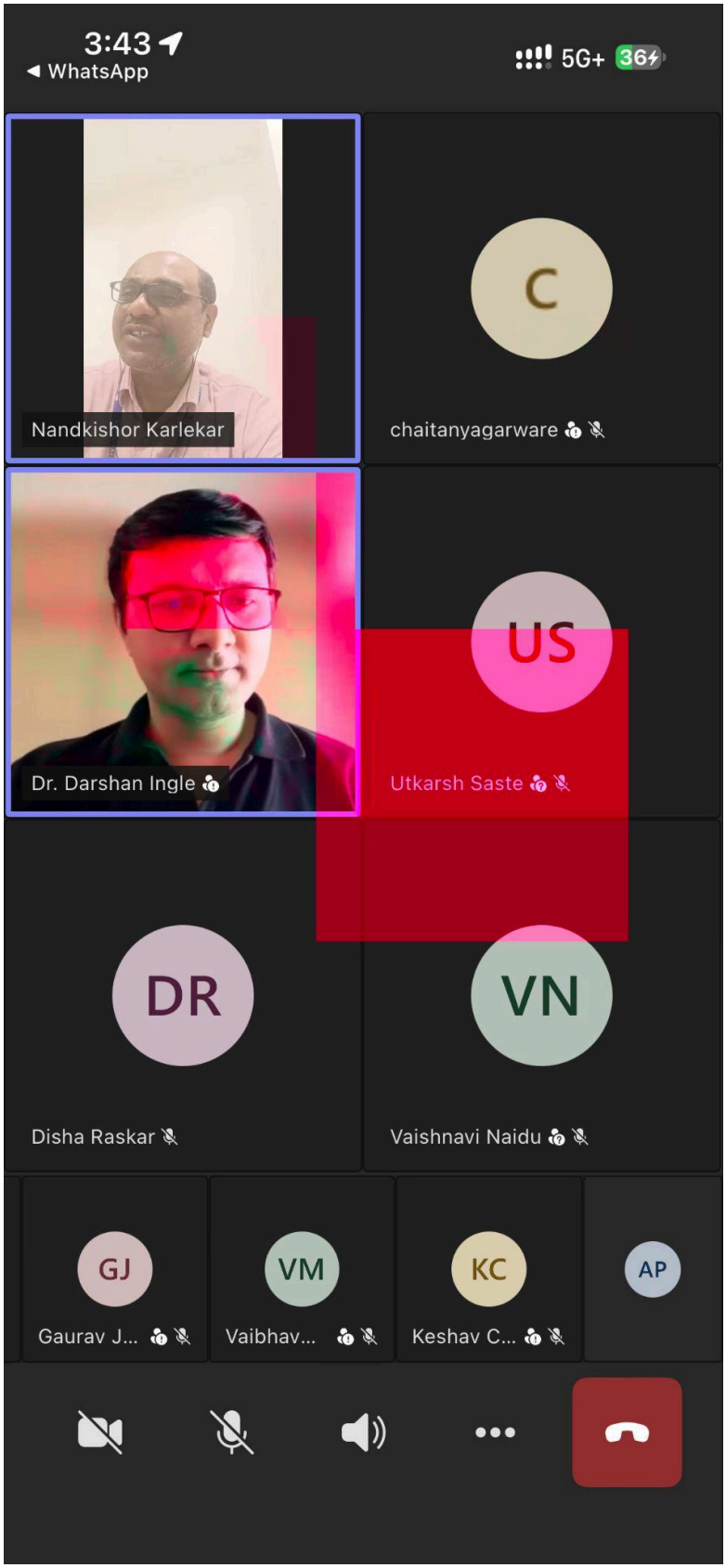
- Learned how applications, jobs, stages, tasks and partitions fit together in Spark's execution process.
- Understood how executor-core availability influences the number of tasks that can run simultaneously.
- Recognized that transformations are evaluated lazily and that actions initiate computation.
- Practised the purpose of filtering rows, selecting columns, creating derived columns and aggregating grouped data.
- Identified the memory risk of collecting an entire large dataset and the overhead that shuffles can introduce.
- Gained a clearer understanding of why query plans, early filtering and sensible partitioning matter for performance.

10. Conclusion

The guest lecture provided a useful view of how modern big-data processing works beyond the level of individual commands. The explanations and notebook demonstrations showed how Spark divides work, delays computation until necessary, and applies optimizations to process data efficiently. The session strengthened my understanding of distributed data engineering and highlighted practical habits for building more scalable ETL pipelines, especially controlling driver-side data collection and minimizing avoidable shuffles.

Submitted by: Adesh Patankar

Subject: BDTL


3:43
WhatsApp
5G+
Nandkishor Karlekar
C
chaitanyagarware
Dr. Darshan Ingle
US
Utkarsh Saste
DR
Disha Raskar
VN
Vaishnavi Naidu
GJ
Gaurav J...
VM
Vaibhav...
KC
Keshav C...
AP